

Automated Investment Portfolio & Yield Analyzer

A Comprehensive Problem Statement on C++ Abstract Classes, Pure Virtual Functions, Dynamic Memory Allocation, and Runtime Polymorphism

Key Learning Objectives:

- Define and enforce Abstract Base Classes with Pure Virtual Functions (`= 0`).
- Implement dynamic multi-tier object hierarchies and polymorphism using base pointers.
- Manage runtime dynamic arrays of pointers (`Type**`) using manual heap allocation (`new[]`) and deallocation (`delete[]`).
- Prevent memory leaks through virtual destructors and clean dynamic deallocation loops.
- Model real-world financial yield calculations, volatility benchmarks, and weighted risk metrics.

1. Problem Background & Domain Overview

Modern wealth management platforms allow retail and institutional investors to construct diversified portfolios composed of heterogeneous financial assets. In this problem, you are tasked with developing the core analytics engine for a portfolio management system in C++.

The system supports three major investment classes: Equity Stock, Real Estate Investment Trust (REIT), and Crypto Staking. Although all holdings share common metadata (such as an asset ticker symbol, asset name, and initial invested principal), each asset class operates under distinct financial mechanics for projecting returns, accounting for management overhead, and quantifying risk.

Crucially, the number of holdings (N) in any given investor's portfolio is dynamic and unknown at compile time. Your solution must dynamically allocate memory on the heap and leverage dynamic polymorphism to evaluate and report on the portfolio.

2. Object-Oriented Class Specifications

2.1 Abstract Base Class: `InvestmentAsset`

The base class serves as an uninstantiable abstract contract defining core properties and pure virtual interfaces.

- **Protected Attribute:** `string assetTicker` (e.g., 'AAPL', 'VNQ', 'ETH')
- **Protected Attribute:** `string assetName` (Full name of holding)
- **Protected Attribute:** `double initialPrincipal` (Invested amount in USD)
- **Constructor:** `InvestmentAsset(string ticker, string name, double principal)` to initialize common data members.

- **Virtual Destructor:** `virtual ~InvestmentAsset()` — MUST be declared virtual to ensure proper polymorphic deallocation of derived objects through base pointers.
- **Pure Virtual Method 1:** `virtual double calculateProjectedValue(int durationYears) = 0` — Computes and returns the future compound value of the asset.
- **Pure Virtual Method 2:** `virtual double calculateRiskScore() = 0` — Computes and returns a standardized risk rating on a scale from 1.0 (lowest) to 10.0 (highest).
- **Pure Virtual Method 3:** `virtual void displayStatement(int durationYears) = 0` — Formats and prints a detailed holding summary, projected value, and risk breakdown to the console.

2.2 Concrete Derived Class: EquityStock

- **Private Attribute:** `double annualGrowthRate` (Expected capital gain per year, e.g., 0.08 for 8%)
- **Private Attribute:** `double dividendYield` (Annual dividend rate, e.g., 0.02 for 2%)
- **Private Attribute:** `double beta` (Market volatility coefficient relative to the S&P 500)
- **Financial Logic:** $\text{Projected Value} = \text{initialPrincipal} * (1 + \text{annualGrowthRate} + \text{dividendYield})^{\text{durationYears}}$
- **Risk Formula:** $\text{Risk Score} = \min(10.0, \max(1.0, \text{beta} * 4.5))$

2.3 Concrete Derived Class: RealEstateTrust (REIT)

- **Private Attribute:** `double rentalCapRate` (Gross rental yield per year)
- **Private Attribute:** `double propertyAppreciationRate` (Estimated real estate capital growth)
- **Private Attribute:** `double managementFeeRate` (Annual property management deduction)
- **Financial Logic:** $\text{Net Annual Rate (r)} = \text{rentalCapRate} + \text{propertyAppreciationRate} - \text{managementFeeRate}$.
 $\text{Projected Value} = \text{initialPrincipal} * (1 + r)^{\text{durationYears}}$
- **Risk Formula:** Risk Score = Fixed at 3.5 (Moderate real estate market stability)

2.4 Concrete Derived Class: CryptoStaking

- **Private Attribute:** `double stakingAPY` (Annual Percentage Yield from validator consensus)
- **Private Attribute:** `double networkSlashingPenaltyRate` (Risk penalty deduction for network/validator downtime)
- **Financial Logic:** $\text{Effective Yield (y)} = \text{stakingAPY} - \text{networkSlashingPenaltyRate}$. $\text{Projected Value} = \text{initialPrincipal} * (1 + y)^{\text{durationYears}}$
- **Risk Formula:** Risk Score = Fixed at 8.5 (High volatility, smart contract and liquidity risk)

3. Class Architecture Summary

Class Name	Inheritance	Specific Data Members	Risk Formula / Metric
EquityStock	public InvestmentAsset	annualGrowthRate, dividendYield, beta	$\min(10.0, \text{beta} * 4.5)$
RealEstateTrust	public InvestmentAsset	rentalCapRate, propertyAppreciationRate,	Fixed 3.5 (Moderate)

managementFeeRate			
CryptoStaking	public InvestmentAsset	stakingAPY, networkSlashingPenaltyRate	Fixed 8.5 (High)

4. Dynamic Memory & Main Function Workflow

A primary requirement of this exercise is the strict management of runtime dynamic memory using raw pointers. The program must follow these structured steps in main():

- 1. Runtime Size Input: Prompt the user to enter the number of portfolio assets (N) and the investment duration in years (T).
- 2. Dynamic Base Pointer Array Allocation: Allocate an array of base class pointers on the heap:

```
int N, durationYears;
cout << "Enter number of assets: "; cin >> N;
cout << "Enter investment horizon (years): "; cin >> durationYears;

// Allocate dynamic array of base pointers
InvestmentAsset** portfolio = new InvestmentAsset*[N];
```

- 3. Interactive Factory Loop: Iterate from index 0 to N - 1. For each asset, prompt the user for the asset type (1: Equity Stock, 2: Real Estate Trust, 3: Crypto Staking) and instantiate the corresponding derived class dynamically using 'new'.
- 4. Polymorphic Report Generation: Loop through the array using base class pointers to execute displayStatement(durationYears), calculateProjectedValue(durationYears), and calculateRiskScore().
- 5. Aggregate Financial Metrics: Calculate the total initial capital invested, total projected future portfolio value, aggregate profit/loss, and portfolio-weighted average risk score.
- 6. Clean Dynamic Deallocation: Iterate through the array to deallocate each derived object, followed by freeing the pointer array itself to avoid memory leaks:

```
// Clean deallocation
for (int i = 0; i < N; ++i) {
    delete portfolio[i]; // Invokes virtual destructor
}
delete[] portfolio;      // Frees the pointer array
portfolio = nullptr;
```

5. Sample Program Execution & I/O Trace

```
=== AUTOMATED INVESTMENT PORTFOLIO & YIELD ANALYZER ===
Enter number of assets (N): 3
Enter investment duration horizon (years): 5
```

```

--- Asset #1 Entry ---
Select Type (1: Equity Stock, 2: REIT, 3: Crypto Staking): 1
Enter Ticker: AAPL
Enter Asset Name: Apple Inc.
Enter Initial Principal ($): 10000
Enter Expected Annual Growth Rate (e.g. 0.10 for 10%): 0.10
Enter Annual Dividend Yield (e.g. 0.015 for 1.5%): 0.015
Enter Beta: 1.2

--- Asset #2 Entry ---
Select Type (1: Equity Stock, 2: REIT, 3: Crypto Staking): 2
Enter Ticker: VNQ
Enter Asset Name: Vanguard Real Estate ETF
Enter Initial Principal ($): 15000
Enter Rental Cap Rate: 0.05
Enter Property Appreciation Rate: 0.03
Enter Management Fee Rate: 0.01

--- Asset #3 Entry ---
Select Type (1: Equity Stock, 2: REIT, 3: Crypto Staking): 3
Enter Ticker: ETH-STK
Enter Asset Name: Ethereum Validator Staking
Enter Initial Principal ($): 5000
Enter Staking APY: 0.07
Enter Network Slashing Risk Penalty: 0.005

===== PORTFOLIO PERFORMANCE REPORT =====
[Equity Stock] AAPL - Apple Inc.
  Initial Principal : $10,000.00
  Projected (5 Yrs) : $17,233.54 (Net Growth: 72.34%)
  Risk Score       : 5.40 / 10.00 (Moderate-High Volatility)
-----
[Real Estate] VNQ - Vanguard Real Estate ETF
  Initial Principal : $15,000.00
  Projected (5 Yrs) : $20,073.38 (Net Growth: 33.82%)
  Risk Score       : 3.50 / 10.00 (Low-Moderate Volatility)
-----
[Crypto Staking] ETH-STK - Ethereum Validator Staking
  Initial Principal : $5,000.00
  Projected (5 Yrs) : $6,850.43 (Net Growth: 37.01%)
  Risk Score       : 8.50 / 10.00 (High Risk/Yield)
=====
PORTFOLIO AGGREGATE SUMMARY:
  Total Invested Principal : $30,000.00
  Total Projected Value    : $44,157.35
  Total Projected Profit   : $14,157.35 (+47.19%)
  Weighted Avg Risk Score  : 4.97 / 10.00
=====
Memory cleaned up successfully. 3 asset instances deallocated.

```